Appendix A

Eigenclass Formalization

The definition of **eigenclass** that we use throughout the textbook was an inexact concept long after the first draft. After some deeper consideration of the problem and discussions with colleagues, I propose the following thought experiments to assist us in creating a suitable definition.

Consider the following potential function of a particle traveling in a 1-dimensional chamber. Each potential well is defined by a Gaussian function:

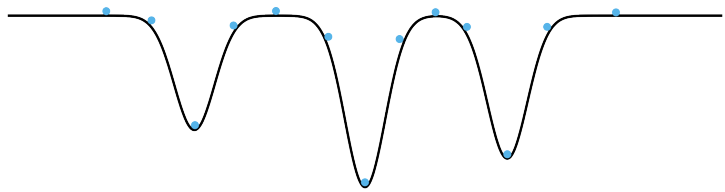


Figure A.1: A potential function with 3 minima that may be found by derivatives via an optimizer

It is easier for the particle to crawl down a potential well than up, but it is still possible. There are three potential minima that a particle can tend towards, and we can determine this using the second derivative. Any optimizer will find any of the three minima without issue.

Consider a similar potential function in which we replace each Gaussian function with a sombrero function. There are still three major potential wells, but there are many minor potential wells surrounding them due to the bumpiness of a sinc function:

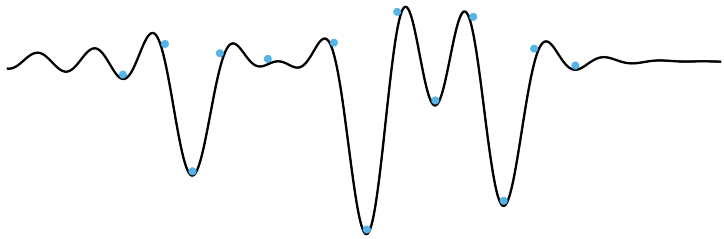


Figure A.2: The same potential function with various high frequency bumps

The major potential minima are obvious to a human eye but not obvious to an optimization algorithm. We cannot rely on a second derivative optimizer because it may be trapped in a nearby local minima.

If we were to rely on Fourier analysis, we could remove the high frequencies from the minima due to their slow decay in the heat equation. We can easily declare the most important minima using our eyes, but that is dependent upon our subjective cognition. More importantly, the strategy does not scale to higher dimensions or discrete optimization problems. Is there a mathematical way to amend these problems?

Consider the following model of gas particles in a 2D chamber, mimicking five chemistry flasks connected together:

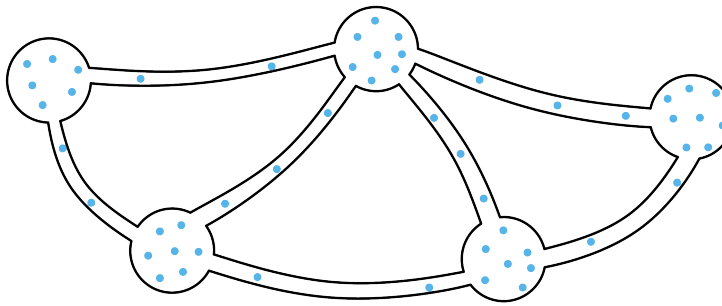


Figure A.3: A representation of 5 gas chambers in 2D

The gas particles flow freely within each flask and along tubes, but they may not pass through the glass walls. From the perspective of a lone gas

particle, how do we know that there are precisely 5 chambers? How does our human vision system designate that there are 5 countable chambers?

How does this relate to Boolean statistical mechanics and eigenclasses? When we are looking at a communication class in a Markov chain, we are interested in how many states can reach each other. As time passes, all states within a communication class have diffused across the entire communication class.[1] They reach a stationary distribution, and all initial distributions in that communication class point in the one direction. Before they all become one, all states are orthogonal at time $t = 0$ as represented by the stochastic matrix $P^0 = I$.

Define $\Theta(t)$ to be some matrix such that $\Theta_{(i,j)}$ represents the correlation or cosine similarity between row i and row j of the stochastic matrix P^t:

$$\Theta_{(i,j)}(t) = \frac{P^t_{(i,\cdot)} \cdot P^t_{(j,\cdot)}}{||P^t_{(i,\cdot)}||||P^t_{(j,\cdot)}||}$$

At time $t = 0$, the correlation value for non-diagonal entries is 0 for all stochastic matrices. As time $t \to \infty$, the correlation between each state is 1 within an ergodic communication class. Thus we define $\Theta(t)$ for all times values $t \in \mathbb{N}$.

Let us define $k(t)$ to be the sum of all values in the matrix $\Theta(t)$:

$$k(t) = \frac{n^2}{\sum \Theta(t)}$$

Since P is some $n \times n$ matrix, $k(0)$ is always n since any state is correlated with itself and $k(\infty)$ has a minimum value of 1 for most stochastic matrices. We define $k(t)$ as the **pseudo-rank** of the correlation matrix of stochastic matrix P at time t.

The stochastic matrix at time $t = 0$ is the identity matrix $P^0 = I$. The state space of P^t is defined by some $(n-1)$-simplex surface, and each row must be a point on the simplex. As $t \to \infty$, the estimated number of dimensions compresses towards 1 if the stochastic matrix diffuses across all states. Somehow, the estimated number of dimensions compresses from n to 1 due to the fact that the eigenvalues of a stochastic matrix are bounded by $|\lambda| \leq 1$.

We also define $m = \lceil k(t) \rceil$ to be the minimum number of rows and columns needed to sufficiently represent P^t as a lower-order matrix Q:

$$P^t_{n \times n} \approx Q_{m \times m}, \quad m = \lceil k(t) \rceil,\ m << n$$

[1] Except for the odd case of periodic Markov chains, in which case the initial states may be separated by phase in a cycle.

The order of matrix P^t is approximately m. At the limit of infinite time steps, the value for $\lceil k(\infty) \rceil$ equal to the number of recurrent classes times the period of each class.[2]

If we think of the cyclical stochastic matrix with two different communication classes:

$$P = \left[\begin{array}{cc|ccc} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 0 \end{array}\right]$$

then $\lceil k(t) \rceil \to \frac{25}{2+3}$ as $t \to \infty$. The estimated number of classes is exactly 5, therefore we need Q to be a rank 5 matrix to sufficiently approximate P^∞.

If we think of a matrix that will perfectly diffuse but has two communication classes:

$$P = \left[\begin{array}{cc|ccc} \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \hline 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{array}\right]$$

then have $\lceil k(t) \rceil \to \lceil \frac{25}{4+9} \rceil$ as $t \to \infty$. The estimated number of classes is approximately 1.923, we need Q to be at least a rank 2 matrix to sufficiently approximate P^∞.

The value that this provides to Boolean statistical mechanics is that the number of potential wells for a stochastic matrix P is much less obvious than a potential function. We cannot rely on the derivative of $k(t)$ to determine an optimal number of classes because $\frac{d}{dt}k(t)$ relates to the eigenvalues $\lambda_1, ..., \lambda_\theta$ of P. Since each λ^t is an exponential function of C^ω analytic smoothness, the sum of correlations $k(t)$ is likewise a C^ω smooth functions. This means we should rely on the elbow method to define the number of classes, such as if we use the k-means clustering algorithm.[3]

[2] If d_i is the period of class i, then $\lceil k(\infty) \rceil = \sum d_i$ is the number of unique limiting distributions.

[3] The elbow method in cluster analysis is often decried as dependent upon human subjectivity. Now we have an analytic reason of why derivatives do not work in cluster analysis.

As initial states in stochastic matrix P^t become more correlated, initial states with similar future distributions merge. We might think of eigenclasses as a vertical slice of some tree diagram as time progresses:

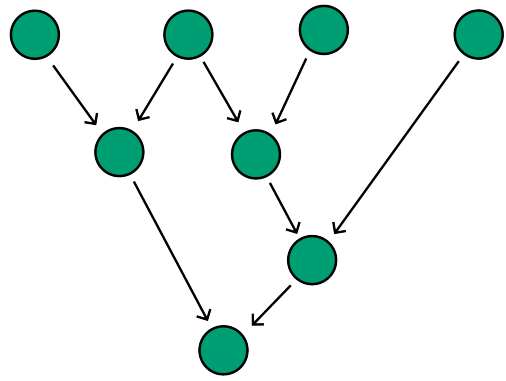


Figure A.4: The merging of initial distributions with similar future distributions form a taxonomic class or category

Suppose that we do a discrete approximation of the gas chambers example. If we use the elbow method to stop at $\lceil k(t) \rceil = 5$ classes the optimal classification of initial points looks something like this:

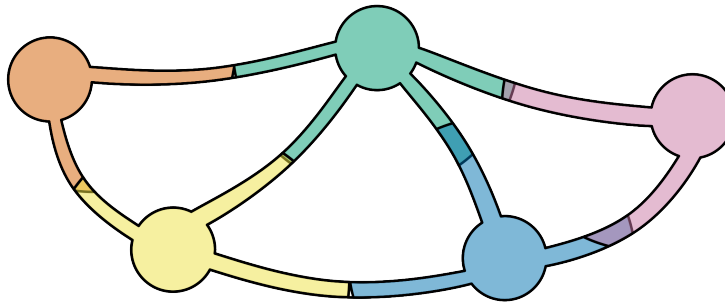


Figure A.5: The five classes of a gas chamber are due to "bottlenecks" that restrict flow

The probability distribution of a single gas particle will easily spread within its chamber, but how quickly it spreads will *bottleneck* due to the thin tubes

connecting the gas chambers. We might think of it as a sharp decrease in the rate of unions, fusions, or anti-bifurcations occurring in the stochastic matrix.

One possible method for determining a temporal class is ***k*-means clustering**. A k-means clustering algorithm seeks to assign n points to k clusters by minimizing the average distance to a cluster, and this is useful for the context of determining where to assign rows of a stochastic matrix based upon their future distribution:

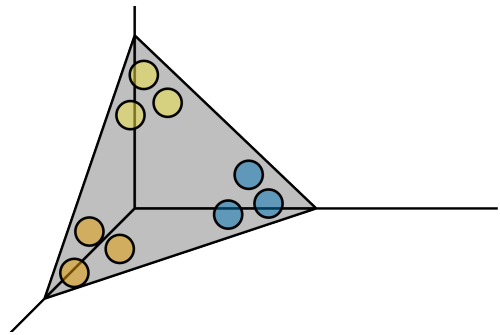


Figure A.6: A 2-simplex as the projection of 9 states into 3 classes due to clustering

In this example, there are approximately $\lceil k(t) \rceil = 3$ possible clusters that a point may be assigned to.

The flow of these n points must flow upon some $(n-1)$-simplex as governed by the eigenvalues of the stochastic matrix P^t. We need at most some $(m-1)$-simplex for our projection into a lower-order matrix, in which $m = \lceil k(t) \rceil$. This is a form of **dimensionality reduction** for stochastic matrices, at it is based on the idea that the determinant $|P^\infty| = 0$. While k-means clustering is a decent naive assumption, we need an algorithm that leverages the unique property that a probability distribution for each row of P^t is bound to the $(n-1)$-simplex and may be projected to some $(m-1)$-simplex. [4]

[4] The triangle inequality for cosine similarity is sometimes used to accelerate k-means algorithms.

Appendix B

Approximate Dynamical Systems

Dynamical systems are of great interest to many systems we wish to predict such as fluids, neuronal systems, robotic control systems, and videos. Most systems in the real world are governed by non-linear dynamics, but we are interested in linear systems of the form:

$$\frac{d}{dt}f(x) = Af(x)$$

in which A is a matrix.

Most measure spaces (mass, energy, velocity) will be governed by non-linear dynamics. However, probability measure spaces will *always* be governed by linear dynamics:

$$\mu(t+1) = \Lambda_\sharp \mu(t) = P\mu(t)$$

This means that we should consider approximations of the form:

$$f(t+1) \approx \sigma^{-1}\tilde{P}\sigma f(t)$$

The key assumptions for why this work is that are projecting some measure space into a probability measure space using the function σ, and we can approximate P with $\tilde{P}$:

$$\sigma f(t) \approx \mu(t), \quad \tilde{P} \approx P$$

The pushforward form of our projection is thus:

$$\sigma f(t+1) \approx \tilde{\Lambda}_\sharp \sigma f(t) \approx \tilde{P}\sigma f(t)$$

We can approximate any non-linear, non-conservative systems because probability is always conserved. What is necessary to optimally approximate any dynamical system? If a dynamical system has n eigenvalues $\lambda_1, ..., \lambda_n$, then using the m largest magnitude eigenvalues $|\lambda_1|, ..., |\lambda_m|$ will minimize the accumulation of prediction error for any time-dependent measure.

Appendix C

Graph Minor Algorithms

Edge contraction is a way that we may represent collections of states (or macrostates). By merging two vertices u, v connected by an edge, a contracted vertex w is formed containing all edges from both of the original vertices (u, v). **Graph minors** are graphs formed by deleting edges, deleting vertices, and contracting edges:

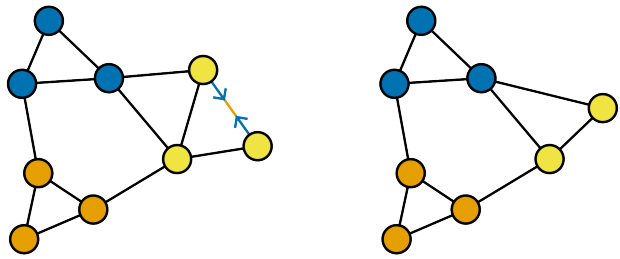


Figure C.1: A graph and its graph minor via edge contraction

The graph minor g of the graph G represents a reduced set of information about the original graph.

Classes define states that strongly communicate with each other. When states in a Markov chain cluster on a simplex due to similar futures, we contract their edges in the state space $\mathcal{X}$ to form a macrostate. Each macrostate defines a corner of a reduced dimension simplex. What does that mean? What about graphs not representing Markov chains? Without some way to measure some value of a graph and its minor graph, there is no objectivity in determining how to contract edges.

Assume that every graph is a directed graph with self-loops and may represent a Markov chain:

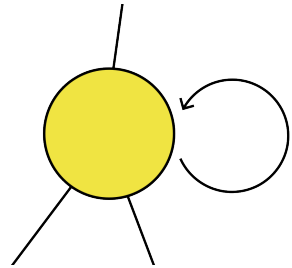


Figure C.2: A directed graph with self-loops is a necessary assumption for merging via aperiodic Markov chains

We claim that edges of a graph should contract to merge vertices if and only if they communicate strongly. We create the assumption that every graph is a stochastic matrix for an aperiodic Markov chain.

Observe the following example of some graph representing 5 states:

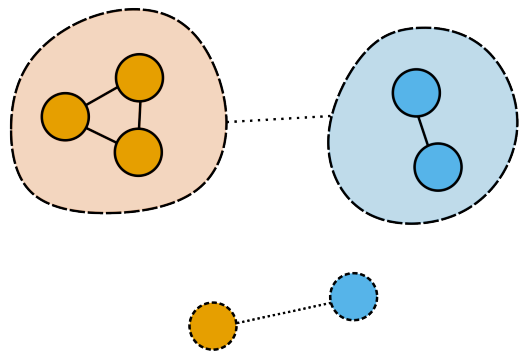


Figure C.3: The five microstates can be abstracted as two macrostates as if they formed a reducible Markov chain

We say that the solid lines indicate states that communicate strongly and dotted lines indicates that they communicate weakly. The dashed lines around a cluster is an abstraction of a larger state that is porous.

For a transition matrix P, we can abstractly represent the communication classes by separating the matrix into blocks. The diagonals blocks represent communication classes, and the non-diagonal blocks represent

transitions between classes:

$$P^{\tau} = \left[\begin{array}{cc|ccc} \frac{1}{2} & \frac{1}{2} & \varepsilon & \varepsilon & \varepsilon \\ \frac{1}{2} & \frac{1}{2} & \varepsilon & \varepsilon & \varepsilon \\ \hline \varepsilon & \varepsilon & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \varepsilon & \varepsilon & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \varepsilon & \varepsilon & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{array}\right] \rightarrow P^{\tau} = \left[\begin{array}{c|c} P_1 & \varepsilon \\ \hline \varepsilon & P_2 \end{array}\right]$$

In this example, the determinant within each class is approximately 0 some finite time τ. The communication strength between classes is some infinitesimal transition probability ε. What we want to say about what defines a class is that the determinant of the matrix in each partition is approximately 0 after some finite amount of time.

Appendix D

Class Stochastic Matrices

A **class stochastic matrix** is a stochastic matrix $Q_{m\times m}$, and it is the dimensionality reduction of stochastic matrix $P_{n\times n}$ at time t such that:

$$Q = K^{+}P^{t}K$$

We define K as the classification matrix of size $n \times m$ and K^{+} as the Moore-Penrose pseudo-inverse of K of size $m \times n$. We may disambiguate P as the state transition matrix or state stochastic matrix.

A **classification matrix** K is a right-stochastic matrix that compresses the n prime states of the stochastic matrix P^t into m composite states of the stochastic matrix Q. The classification matrix K is a $(n \to m)$ projection of the state probability vector $y_{1\times n}$. We may therefore define the class probability vector as yK. We may call K^{+} the decompression matrix.[1] An optimal compression retains the m largest eigenvalues of $\Lambda^t = \{\lambda_1^t, ..., \lambda_n^t\}$. Non-optimal compression does not retain these slowly decaying eigenvalues.

In terms of machine intelligence, the optimal compression of a dynamical system uses the longest lasting eigenvalues because it will minimize the divergence of predictions in the state space over time. In contrast, the superfluous information contained in fast decaying eigenvalues is washed away with few time steps. Optimal compression must use the largest eigenvalues to minimize the cumulative prediction error, and they do so for *any* error metric.

The name class stochastic matrix is important. The stochastic matrix of a random walk eventually converges onto a uniform distribution, but it first converges to be approximately uniform within each diffusion class. Similar to how a reducible Markov chain has multiple communication classes, a class stochastic matrix has multiple classes that weakly com-

[1] It is not accurate to call K^{+} the declassification matrix. It inexactly separates each class into its component states, but information is irreversibly lost when passing through K akin to information entering a black hole.

municate *between* classes and strongly communicate *within* classes. We also adapt statistical classification algorithms to classify rows of P^t.

The following state transition matrix P has an obvious class structure as a block diagonal matrix:

$$P = \left[\begin{array}{cc|ccc} \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \hline 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{array}\right]$$

We introduce a small jump probability ε, and this ε represents a small, finite value in which the prime states in a composite state weakly communicate with prime states from another composite state.

Here is the corresponding graph visualization of P if we introduce the jump probability ε:

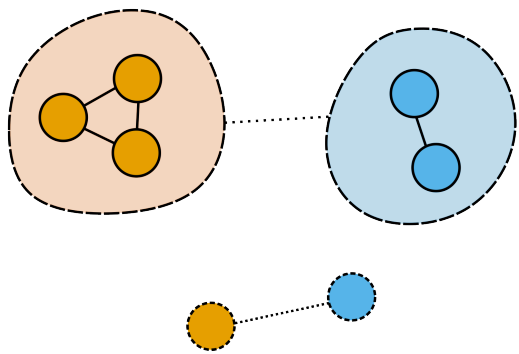


Figure D.1: Each state in a class i has a $1 - \varepsilon$ chance of moving to a state within class and a ε chance of traveling to a state in any other class

The communication gap between two classes is large and within each class is small. We use this jump probability to define a new transition matrix P^*.

We now define our new transition matrix P^* with class exit probability

ε distributed uniformly among states outside of a given class:

$$P^* = \left[\begin{array}{cc|ccc} \frac{1}{2}-\frac{\varepsilon}{2} & \frac{1}{2}-\frac{\varepsilon}{2} & \frac{\varepsilon}{3} & \frac{\varepsilon}{3} & \frac{\varepsilon}{3} \\ \frac{1}{2}-\frac{\varepsilon}{2} & \frac{1}{2}-\frac{\varepsilon}{2} & \frac{\varepsilon}{3} & \frac{\varepsilon}{3} & \frac{\varepsilon}{3} \\ \hline \frac{\varepsilon}{2} & \frac{\varepsilon}{2} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} \\ \frac{\varepsilon}{2} & \frac{\varepsilon}{2} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} \\ \frac{\varepsilon}{2} & \frac{\varepsilon}{2} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} & \frac{1}{3}-\frac{\varepsilon}{3} \end{array}\right]$$

Because the weakly communicating transition matrix an eigenvalue $\lambda_1 = 1$ with uniform eigenvector $\phi_1 = \frac{1}{n}$, all rows of the transition matrix converge to the vector $\frac{1}{n}$ as $t \to \infty$. In the transition matrix with separate communication classes, each class has their own eigenvector corresponding to the uniform distribution within the communication class.

Using our informal intuition and no formal algorithm, we may judge the stochastic matrix P to have the following hard classification matrix K:

$$K = \begin{bmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}$$

A key property of hard classification matrices is that K^+K is precisely the stochastic matrix with m communication classes that are uniformly distributed within each class

$$K^+K = \left[\begin{array}{cc|ccc} \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & 0 & 0 \\ \hline 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{array}\right]$$

We have thus reconstructed the block diagonal matrix from earlier.

Let us suppose that we wish to perform a similar example for a much larger class matrix with the following visualization:

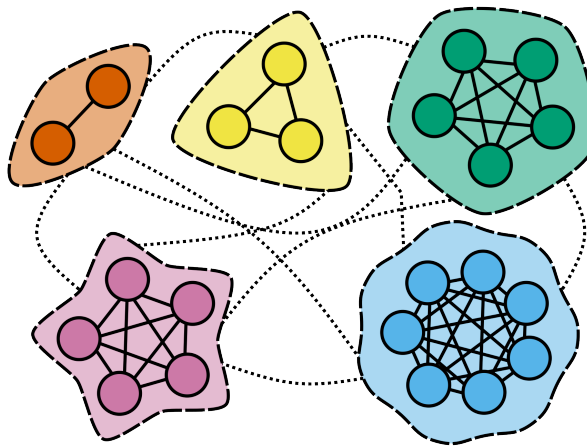


Figure D.2: A system with 22 states in 5 classes that weakly communicate

Each state in a class i has a $1 - \varepsilon$ chance of moving to any state within its class and a ε chance of traveling to a state in any other class. The "obvious" class structure means that we can define the classification matrix K intuitively and without an algorithm.

Let us suppose for the example that we want the class stochastic matrix Q at time $t = 20$. We calculate Q using the formula $Q = K^{+}P^{20}K$. According to the class estimation formula:

$$k(t) = \frac{n^2}{\sum \Theta(t)}$$

the estimated number of classes is $m = \lceil k(20) \rceil = 5$. If we use any other classification matrix K, we lose the property that Q possesses the m largest magnitude eigenvalues of P^{20}. In this example, the computed eigenvalues of P^{20} are:

1.000000e+00, 9.569909e-01, 9.569858e-01, 9.569792e-01, 9.569739e-01, 1.987494e-307, 3.814810e-314

Likewise, the computed eigenvalues of Q are

1.0000000, 0.9569909, 0.9569858, 0.9569792, 0.9569739

It may perhaps seem "obvious" that such a toy example that Q preserves the m largest eigenvalues of P^{20}, but it is not so obvious that it should be the case for transition matrices in general.

There are two kinds of classification matrices K: hard classification and soft classification. A hard classification matrix K may only allow Boolean values $\mathbb{B}_\circ = \{0, 1\}$. In contrast, a soft classification matrix K may allow any value from the set $[0, 1]$ as long so the row sums to 1. A hard classification matrix is preferable to a soft classification matrix because a microstate should completely diffuse across its macrostate before two macrostates are ready to merge. Any precision gained by using a soft classification matrix is lost if a microstate is uniformly distributed within its class in due time. Hard classification is essentially a function of human language, meaning that words are required to decompress a general sentence into more detail. Language structure depends upon a latent contraction hierarchy.

The intuitionist justification for hard classification is that the **partial determinant** of P^t for some (i, j) is approximately 0 if these state belong in the same class:

$$\begin{vmatrix} (P^t)_{(i,i)} & (P^t)_{(i,j)} \\ (P^t)_{(j,i)} & (P^t)_{(j,j)} \end{vmatrix} \approx 0$$

The diagonal entries represent self-flow and the off-diagonal entries represent cross-flow. States in the same class continuously flow and diffuse between each other until they are indistinguishable.

The name of classification matrix is also appropriate because of the hierarchical tree structure similar to taxonomy, the science of classification:

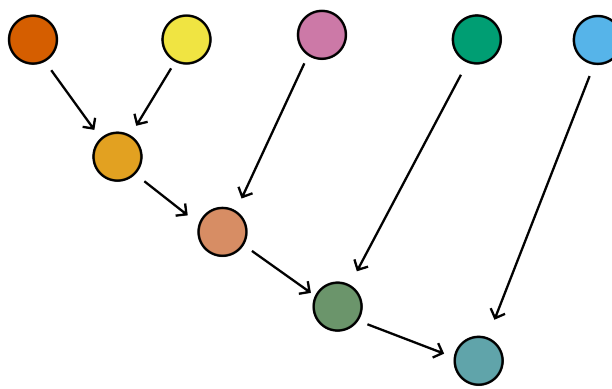


Figure D.3: A class merges all microstates into a single macrostate

A scientific taxonomy starts with a general class and splits into a greater number of specific classes. In contrast, a contraction hierarchy of the state

space $\mathcal{X}$ starts with specific classes and merges into fewer number of general classes.

This begs the question: What algorithm determines the optimal classification structure matrix K that results in Q having the m largest eigenvalues of P^t? There are a few methods I recommend:

1. k-means Clustering
2. k-means by Correlation
3. Correlation Merge

Many programming languages have k-means clustering available as a coding package. My k-means by correlation algorithm is an adaptation of k-means that leverages the triangular structure of the simplex, balancing speed and accuracy. Correlation merge is deterministic and slow in finding the global optima, but its cardinal sins are being obvious, boring, and uncreative.

The ***k*-means clustering** algorithm is a classification algorithm that assigns data points to a class depending upon their distance to a class average.

1. Initialize k points to be the center of a class.
2. Assign a class each point depending on the nearest class center by Euclidean distance.
3. Calculate the new center of each class by the average value of assigned members.
4. Iterate until convergence or until maximum iterations have run.

This algorithm works reasonably well for classifying correlated rows of a transition matrix P. However, it may fail to converge to the optimal cluster for any given data set depending on how the k points are initialized. It implicitly minimizes the sum of square distances within each class, and the problem of minimizing overall sum of squares is **NP**-hard computational complexity. Thanks to support from mathematics academia, k-means is available as a package in common coding languages.

The ***k*-means by correlation** algorithm is a classification algorithm that assigns data points to a class depending upon their correlation to a class expectation. Since we know that the rows of a transition matrix P^t are bound to the $(n-1)$-simplex for $t \in \mathbb{N}$, correlation best exploits the triangular structure.

1. Initialize k points to be the center of a class.
2. Assign a class each point depending on the nearest class center by correlation.
3. Calculate the new center of each class by the average value of assigned members.
4. Iterate until convergence or until maximum iterations have run.

Notice the similarity to k-means clustering. It may likewise fail to converge to the optimal class assignment, and the primary issue is accidentally merging two classes and forcing one empty class. Trials of the algorithm on the Markov chain represented in *Figure D.2* often merges the smallest classes but almost never the large class.

This can be easily circumvented by doubling the amount of possible classes to $2k$, then cutting the algorithm runtime shortly after we count k non-empty classes.[2] This finite runtime after reaching k classes is called the refinement period, we repair empty classes by randomly generating a new point within the simplex.[3] I call these modifications of doubling, counting, refining, and repairing to create the "shotgun method" for k-means by correlation. We discard all empty classes when constructing the classification matrix K. This is rather easy to code, maintaining the speed of k-means while being almost as accurate as correlation merge.

Merging rows by correlation, or **correlation merge** is likely the best deterministic method since the intuition is that classes are defined by correlation. Expect it to be rather slow however. There are a few different ways that we can merge rows of P^t:

1. Merge rows if all of their mutual correlations is greater than some constant.
2. Merge rows if any two have a mutual correlation greater than some constant.
3. Merge rows by strongest correlations until we reach m classes.

[2] We may try k^2 or 2^k classes so long as they are less than the number of original points n, but it is unnecessary.

[3] The refinement period gives us a finite history to check repairs usurping classes. If our repair vector lands inside of an established class, the class will split in two. The algorithm will push one of the redundant vectors out.

We defined the matrix $\Theta(t)$ as the cosine similarity between between the i-th and j-th row of P^t, and we defined:

$$m = \left\lceil \frac{n^2}{\sum \Theta(t)} \right\rceil$$

as the estimated number of classes. Triangle inequality is how we justified the dimensionality reduction of n states to m classes in the first place. If two points A, B share an implied class center C, then triangle inequality places good bounds on how dissimilar A and B can be if each is highly correlated with C.

The covariance matrix Σ for a population sample $X_1, ..., X_n$ may be used to derive the correlation θ between any element X_i and its sample average $\bar{X}$:

$$\theta(X_i, \bar{X}) = \frac{\mathrm{E}[\Sigma_{(i,\cdot)}]}{\mathrm{trace}(\Sigma_{(i,\cdot)})}$$

Recall that each row of a transition matrix P is a unit magnitude vector that defines the correlation between input state i at time 0 and output state j at time t, and recall that $\Theta(t)$ defines the correlation in distribution between row i and row j of P^t. We may simplify the correlation as the average of the correlation matrix Θ:

$$\theta(X_i, \bar{X}) = \mathrm{E}[\Theta_{(i,\cdot)}]$$

We may also generally say that the correlation between any state i and another class C is the average of class elements $j \in C$ in the i-th row of the correlation matrix Θ:

$$\theta(X_i, \bar{X}_C) = \mathrm{E}[\Theta_{(i,j \in C)}]$$

We call this the **eigenclass correlation theorem**, and it states the correlation between a row in transition matrix P_i and a set of states $j \in C$ is precisely the average correlation between the i-th row and rows $j \in C$.

Here is the proof of the statement in non-matrix form. The correlation θ between sample component X_i and sample average $\bar{X}$:

$$\theta(X_i, \bar{X}) = \frac{X_i \cdot \bar{X}}{|X_i||\bar{X}|}$$

We split our correlation into a summation of n terms with weight $\frac{1}{n}$:

$$\frac{X_i \cdot \bar{X}}{|X_i||\bar{X}|} = \frac{1}{n} \sum_{j=1}^{n} \frac{X_i \cdot \bar{X}}{|X_i||\bar{X}|}$$

Notice our average $\bar{X}$ can be split in the summation into its original components X_j:

$$\frac{1}{n}\sum_{j=1}^{n}\frac{X_i \cdot \bar{X}}{|X_i||\bar{X}|} = \frac{1}{n}\sum_{j=1}^{n}\frac{X_i \cdot X_j}{|X_i||\bar{X}|}$$

Since our data components have the same magnitude $|X_i| = |X|$ for any i, this means that $|\bar{X}| = |X|$. We may simplify:

$$\frac{1}{n}\sum_{j=1}^{n}\frac{X_i \cdot X_j}{|X|^2} = \frac{1}{n}\sum_{j=1}^{n}\Theta_{(i,j)}$$

Therefore, the correlation of unit magnitude elements is given by:

$$\theta(X_i, \bar{X}) = \mathrm{E}[\Theta_{(i,\cdot)}]$$

We may therefore generalize this correlation to subsets $C \subset \mathcal{X}$ such that:

$$\theta(X_i, \bar{X}_C) = E[\Theta_{(i,j\in C)}]$$

defines the correlation between a state X_i and the subset C, regardless of whether X_i is or is not in C.

For an intuition about the eigenclass correlation theorem, suppose we have three unit-magnitude one-dimensional vectors $X_1 = +1$, $X_2 = +1$, and $X_3 = -1$. The correlation between X and the class average $C = (X_1 + X_2 + X_3)/3$ equals the average of correlation between X_1 and the components of C. Due to triangle inequality, classifications defined by correlation to some latent class vector C means the dissimilarity of any two class elements is bounded by some constant ε_C. The maximum dissimilarity constant may differ between classes.

Some worthwhile research directions are whether or not the maximal interlink vertex (MIV) between two classes are always adjacent to each other. Here is the problem statement:

> MIV(A, B) is the element of class A is the element of class A with the highest correlation to class B.
>
> MIV(B, A) is the element of class B is the element of class B with the highest correlation to class A.
>
> Are MIV(A, B) and MIV(B, A) always adjacent?

The practical application of this question is speeding up the search for a shortest path. The assumption is that these vertices in each class that maximize the cross correlation are the optimal boundary of each class.

AI corporation rely on fear, uncertainty, and doubt (FUD) to monopolize the attention economy. Without a firm theoretical understanding of class stochastic matrices, artificial intelligence models will continue to fall short of the promise of practical application. While I continue my work alone, I hope my ideas inspire more independent researchers to disbelieve the hype and work diligently to improve computer vision, autonomous control systems, and computer graphics.

State spaces of a finite stochastic matrix are often so large that we are working with measurable functions of states instead of the state space graph itself. This means that an artificial intelligence agent or human necessarily struggles when similarity in measure $\mu_i \sim \mu_j$ is not biconditional with similarity in probability distribution $p_i \sim p_j$. It is necessary to find an encoding measure μ' of the direct measure μ such that:

$$\mu'_i \sim \mu'_j \iff p_i \sim p_j$$

in order to properly predict dynamical systems.